

Batched Feedback and the Random-Access Wall in Search-Based Graph Construction

Édgar Chávez

Computer Science Department, CICESE, Ensenada, México

Abstract

Navigable graphs for nearest-neighbor search are built either *incrementally*, each inserted point searching a graph that mutates as construction proceeds, or in *batch* over a fixed substrate, which buys determinism and parallelism at a price in build time. We measure that price and find where it comes from. Instrumenting the build-side distance counters of a tuned Vamana and of PiPNN, and building every system repeatedly in a paired design on one 64-thread machine, we separate build time into *work* (distance evaluations) and *cost per evaluation*. Letting the batch builder’s substrate mutate in B synchronous blocks (FGRAFT) recovers the feedback loop of incremental construction while the graph stays a function of (data, seed, parameters, B): eight trees with batched feedback match thirty-two frozen trees, the build does $0.88\times$ Vamana’s distance work, and coarse batching ($B=8$) suffices. Yet the build takes $1.55\times$ Vamana’s wall-clock, because the mutating substrate costs more per evaluation. Pushing further, we find a wall that no search-based builder crosses: every one of them, incremental or batched, evaluates distances *adaptively*, one dependent random access at a time, and runs $20\text{--}27\times$ below a dense kernel on the same machine, five to six times of it with the data resident in cache. PiPNN’s order-of-magnitude build advantage is that kernel: it evaluates as many distances per point as our beam does, but as fixed-in-advance dense blocks. We show the beam cannot be batched after the fact (the useful density of a lockstep block is $2\text{--}3\%$), state the wall as a two-ceiling roofline, and delimit its scope: it binds whenever the distance is a black box or the evaluation order is data-dependent, and it is absent only for decomposable distances evaluated on pair sets fixed in advance.

Email address: `elchavez@cicese.edu.mx` (Édgar Chávez)

Keywords: similarity search, approximate nearest neighbor, navigable graphs, parallel index construction, determinism, rooftop

1. Introduction

GRAFT [1] removes mutation from search-based graph construction. It builds a forest of T spatial-approximation trees [2], unions their edges into a fixed, redundant *scaffold*, and *harvests*: every point, in parallel, beam-searches the frozen scaffold with itself as the query, and the vertices that search expands are occlusion-pruned into its adjacency list. The construction is embarrassingly parallel and bitwise deterministic at any thread count, and at matched recall its graphs are within a few percent of tuned Vamana [3, 4] in distances per query. It is also slower to build. This paper asks where that time goes and finds two answers at two different scales.

The first answer is algorithmic and partly good news. A frozen substrate must be routable, at full scale, for the first point it serves, and that routability is bought with redundancy: more trees. On GloVe the frozen design needs $T=32$, and the trees alone cost 75% of what Vamana spends in total. An incremental builder never pays this: every insertion improves the substrate the next one searches. We show that this feedback loop is separable from the serial dependency chain that produces it. FGRAFT mutates the substrate in B synchronous blocks; within a block it is frozen and the writes are disjoint, so the graph remains a pure function of (data, seed, parameters, B). Measured with a paired, repeated-build protocol on one 64-thread machine (§4), eight trees with batched feedback reach the quality of thirty-two frozen trees, the build performs $0.88\times$ Vamana’s distance work, $B=8$ captures the whole gain, and the build time drops to $0.58\times$ that of the frozen quality profile. It does *not* drop below Vamana’s: $1.55\times$ [1.55, 1.56], because the mutating, denser substrate costs $1.8\times$ per distance evaluation where the frozen one costs $1.3\times$.

The second answer is architectural and is the contribution we consider lasting. Chasing the remaining factor, we measured what a search-based builder can do at all (§5). Every such builder, incremental (HNSW, Vamana) or batched (GRAFT, FGRAFT), evaluates distances *adaptively*: which vector is touched next depends on the last result. Each evaluation is therefore a dependent random access, and its rate is bounded twice, by the per-access bookkeeping that adaptivity forces and by the random-access bandwidth of the memory that holds the data. On our machine two independently

engineered search builders sit within $1.3\times$ of each other and $20\text{--}27\times$ below a dense GEMM, and five to six times of that gap is already present with the data in cache. PiPNN [5], whose build is $14\times$ faster than Vamana’s, evaluates as many distances per point as our beam does; its advantage is that its pair sets are fixed in advance and computed as dense blocks. We then ask whether the beam can be batched after the fact, and measure that it cannot: the useful density of a lockstep block of neighboring points is 2–3%, the beams of adjacent points diverge immediately, and the SAT subtrees that define coherent blocks are not compact in the scaffold.

The wall has a clean statement and a clean scope. *Refining a graph construction whose distance evaluations are not coherent, because the distance is a black box or because the evaluation order is data-dependent, is bounded by random-access throughput, not by arithmetic.* It binds for every metric-space index and for every search-based vector index; it is absent only when the distance decomposes into inner products *and* the pair set is known before it is evaluated. Counting distance evaluations, the natural cost model of metric-space search, mismeasures the vector case by a factor of twenty.

2. Setting and protocol

Systems. GRAFT and FGRAFT are the **fg** binary of the public **graft-ann** repository (release v0.3 plus the mutation patch). Baselines are ParlayANN’s Vamana [4] with the authors’ parameters (GloVe $R=100$, $L=200$, $\alpha=1.0$, two passes; SIFT $R=64$, $L=128$, $\alpha=1.15$, two passes), PiPNN with the same R , L [5], and hnsplib [6] at $M \in \{16, 32\}$, $ef_c=200$. Datasets are ann-benchmarks’ GloVe-100 [7] (1.18M, angular, L2-normalized so that every system solves the same Euclidean problem) and SIFT-128 (10^6) [8]; all systems are graded against the same brute-force answer key.

Machine. A 4-socket Xeon E7-4809 v3 (Haswell, 2.0 GHz, AVX2, 32 cores / 64 threads, 20 MB L3 per socket, 1.5 TB RAM), otherwise idle, with no thermal drift (build-time IQR $\leq 0.5\%$ over ten repetitions).

Cost model. Build cost is reported two ways. *Work* is the number of distance evaluations, read from each system’s own build-side counter: **fg** counts every evaluation; ParlayANN’s Vamana computes the count and we patched one line to print it; for PiPNN we added counters to its three evaluating phases (§5.1). *Time* is each system’s own build timer (data already in RAM), measured with a paired, repeated-build design: ten blocks, every system built once per block

in a seeded random order, ratios reported as Hodges–Lehmann estimates with exact 95% confidence intervals and exact Wilcoxon signed-rank tests over the paired blocks. The quotient of the two ratios is the *cost per evaluation* relative to Vamana. Quality is distances per query at a matched recall target, interpolated between adjacent beam widths ($d@r$); a dash means the sweep did not bracket the target. Local completeness is the fraction of a point’s true 10-NN present in its adjacency list.

3. Batched feedback

Let S_0 be the GRAFT scaffold: the union of T SAT trees’ edges, symmetrized. Partition the points into B blocks by a fixed rule ($p \bmod B$). For $b = 0, \dots, B - 1$: harvest the points of block b against S_b , exactly as GRAFT does (beam search of width ef_h from the tree roots, occlusion prune [3, 9] of the expanded set to at most R neighbors), writing each point’s row into its own slot; then

$$S_{b+1} = S_b \cup \{(p, u), (u, p) : u \in A[p], p \in \text{block } b\}.$$

The final graph is the harvested rows, symmetrized and re-pruned, plus the tree-0 spine. Two design points matter. *Union, never replace*: the substrate needs redundancy and the output needs sparsity; they are different objects, and replacing the scaffold by the pruned graph collapses quality (measured in [1]). *Coarse blocks*: B substrate rebuilds, not n , which is what keeps the scheme synchronous and cheap.

Proposition 1 (Schedule independence, blocked). *Under the determinism rules of [1] (keyed randomness, disjoint writes with canonical merges, a total order on candidates) and a block assignment that is a function of the point identifier alone, the FGRAFT adjacency is a function of $(S, d, \text{params}, \text{seed}, B)$: bitwise identical for every thread count and every scheduling of the parallel loops.*

Proof sketch. Induction on blocks. S_0 is deterministic by the cited contract. Given S_b , every point of block b searches a substrate that does not change during the block and writes only its own row; search and prune are deterministic functions of (S_b, p) . S_{b+1} is a canonical merge of deterministic inputs. No step depends on thread identity, count or interleaving. $\square$

Table 1: GloVe-100, ten paired blocks, 64 threads. Medians; HL = Hodges–Lehmann ratio of build time against Vamana with exact 95% CI (every row $p = 0.002$, 10/10 blocks agree). Quality is distances per query at matched recall, relative to Vamana’s 9,204 / 13,325.

arm	build s	G dist	compl.	$d@0.95$	$d@0.97$	HL time [CI]	work	cost/eval
GRAFT T32/ ef_h 600 frozen	390.6	63.87	0.590	1.021	1.047	2.68 [2.67, 2.70]	2.02	1.33
GRAFT T16/ ef_h 400 frozen	181.3	30.28	0.550	1.103	1.164	1.24 [1.24, 1.25]	0.96	1.29
FGRAFT T8/ef_h400 $B=8$	226.3	27.66	0.570	1.058	1.068	1.55 [1.55, 1.56]	0.88	1.77
FGRAFT T8/ ef_h 400 $B=32$	287.2	28.86	0.574	1.050	1.062	1.97 [1.96, 1.98]	0.91	2.16
Vamana R100/L200 2-pass	145.9	31.56	—	1	1	1	1	1
PiPNN R100/L200	10.4	12.56	—	1.447	1.413	0.071	0.40	0.18
hnsplib M16 / M32	41.2 / 65.9	—	—	—	—	0.28 / 0.45	—	—

Table 2: SIFT-128, same protocol. Every GRAFT-family graph is above recall 0.97 at the bottom of the beam ladder, so quality is matched at 0.98 / 0.99 (Vamana: 1,619 / 2,172).

arm	build s	G dist	compl.	$d@0.98$	$d@0.99$	HL time [CI]	work	cost/eval
GRAFT T4/ ef_h 400 frozen	52.1	5.15	0.483	1.074	1.067	1.02 [0.99, 1.03] [†]	0.47	2.16
GRAFT T16/ ef_h 400 frozen	104.3	12.08	0.498	1.002	0.990	2.01 [2.00, 2.03]	1.09	1.85
FGRAFT T4/ ef_h 400 $B=8$	95.5	8.38	0.496	1.011	1.004	1.85 [1.83, 1.88]	0.76	2.44
FGRAFT T2/ef_h400 $B=8$	89.6	8.03	0.492	1.008	1.011	1.73 [1.72, 1.75]	0.73	2.39
FGRAFT T4/ ef_h 400 $B=32$	122.7	8.72	0.498	1.007	0.997	2.37 [2.35, 2.39]	0.79	3.01
Vamana R64/L128 2-pass	51.7	11.06	—	1	1	1	1	1
PiPNN R64/L128	8.8	9.88	—	1.100	1.042	0.17	0.89	0.19
hnsplib M16 / M32	29.4 / 37.2	—	—	—	—	0.57 / 0.72	—	—

[†]Wilcoxon $p = 0.30$, 5/10 blocks: parity.

The gate is executable: `fg -check-determinism` rebuilds at one and at 64 threads and compares graph hashes. It passes for $B \in \{0, 2, 8, 32\}$, and $B \leq 1$ reproduces the published GRAFT hash, so the frozen path is untouched.

4. Results: work, time, and the cost of an evaluation

Tables 1 and 2 carry four findings.

Feedback substitutes for redundancy. On GloVe, $T=8$ with $B=8$ reaches completeness 0.570 and $d@0.97$ within 2% of the frozen $T=32$ profile, at $0.43\times$ its distance work and $0.58\times$ [0.58, 0.58] its build time. On SIFT, *two* trees with feedback reach the quality of sixteen frozen (parity with Vamana within 1% at both recall targets), where four frozen trees are 7% behind.

The work is now below Vamana’s; the time is not. FGRAFT does $0.88\times$ (GloVe) and $0.73\times$ (SIFT) Vamana’s distance evaluations and takes $1.55\times$ and $1.73\times$ its time. The reason is the last column: the mutating substrate

Table 3: PiPNN’s build distance evaluations by phase (our counters), and the resulting throughput against the search builders of Tables 1–2.

	clustering	leaf GEMM	prune	total G	build s	G eval/s
PiPNN GloVe	3.21	8.49	0.86	12.56	10.35	1.21
PiPNN SIFT	2.20	7.29	0.39	9.88	8.82	1.12
Vamana GloVe / SIFT				31.56 / 11.06	145.9 / 51.7	0.216 / 0.214
GRAFT T16 / T4 frozen				30.28 / 5.15	181.3 / 52.1	0.167 / 0.099
FGRAFT $B=8$ (T8 / T2)				27.66 / 8.03	226.3 / 89.6	0.122 / 0.090

costs $1.8\text{--}2.4\times$ per evaluation against the frozen scaffold’s $1.3\text{--}2.2\times$. Later blocks search a denser graph, the union of the scaffold and every finished row, and its beam is a worse pointer chase. The distance count predicted $\approx 1.4\times$; the measurement says $1.55\times$, with a confidence interval that excludes 1.5.

Coarse batching suffices, and finer batching costs. $B=8$ and $B=32$ are within 1% in quality on both datasets, and $B=32$ is 27–28% slower. What a point needs is a substrate improved by many predecessors, not by its immediate predecessor; eight rebuilds are enough, and each additional rebuild densifies what the remaining points must search.

The regime dependence is a result. Where the frozen scaffold is already cheap (SIFT’s local intrinsic dimensionality is far below GloVe’s [10]; $T=4$ does $0.47\times$ Vamana’s work at time parity), mutation removes little cost; it buys quality. Where the scaffold is expensive (GloVe: $T=32$), mutation removes work and the per-evaluation cost takes half of the gain back. On the decision rule fixed before the campaign, full paper at $\leq 1.5\times$ Vamana with parity quality, the batched design lands just outside on both counts. The next section explains why the missing factor is not an engineering matter.

5. The random-access wall

5.1. PiPNN’s advantage, decomposed

PiPNN [5] builds its graph without searching: a leader-based clustering tree places every point in a few hundred leaves of ≈ 340 points, each leaf is brute-forced with a dense rank-update GEMM, the leaf k -NN graphs are unioned, and an α -prune fixes the degree. Its build is $14\times$ faster than Vamana’s on GloVe and $5.9\times$ on SIFT. Instrumenting the three evaluating phases gives Table 3.

Table 4: Build distance throughput (G evaluations/s, 64 threads) versus working set, GloVe prefixes. GEMM: OpenBLAS $X_{[:,n]}X_{[:,b]}^\top$ with $n b \approx 2 \cdot 10^9$ pairs; its rate rises with n only because the matrix shape improves, reaching ≈ 0.9 TFLOP/s. Read the ratios at $n \geq 2 \cdot 10^5$.

n	MB	GRAFT T16	Vamana	GEMM	GEMM/GRAFT	GEMM/Vamana
20,000	9	0.250	0.291	1.47	5.9	5.1
50,000	22	0.226	0.338	1.48	6.5	4.4
100,000	45	0.193	0.276	2.34	12.1	8.5
200,000	90	0.171	0.238	3.41	19.9	14.3
500,000	224	0.167	0.220	4.54	27.2	20.6
1,183,514	530	0.165	0.216	4.45	27.0	20.6

Two things follow. First, PiPNN’s advantage is only partly less work: on GloVe it evaluates $0.40\times$ Vamana’s distances, on SIFT $0.89\times$, and in both cases about 10^4 per point, the same order as our beam (11.5k at $ef_h 400$). Second, the rest is throughput: 1.1–1.2 G evaluations per second, 5–6 $\times$ the search builders, with 93% of its evaluations inside GEMMs. Each point sits in 30 leaves; PiPNN buys quality with redundant brute force it can afford because the kernel is cheap, and still trails Vamana by 41–45% in distances per query on GloVe.

5.2. Throughput versus working set

If the wall were DRAM bandwidth alone, the search builders’ throughput would collapse when the vectors stop fitting in cache. Table 4 builds prefixes of GloVe from 9 MB (inside one socket’s L3) to 530 MB, and times a float32 GEMM on the same prefix with the same order of pair count.

The search rate does not collapse (Figure 1): from in-cache to DRAM-resident, GRAFT loses 34% and Vamana 26%, gradually. Already in cache, both evaluate one distance per ≈ 200 –250 core cycles, against ≈ 50 for a register-blocked kernel at the efficiency OpenBLAS reaches here. That first 5–6 $\times$ is *adaptivity overhead*: every evaluation sits inside a dependency chain (beam heap, visited stamp, best- R tracker, occlusion test) that must complete before the next vertex can be chosen, so it runs at latency speed wherever the operand lives. The DRAM boundary adds the remaining 1.3–1.5 $\times$. At full n , Vamana runs at 65% and GRAFT at 49% of the bandwidth ceiling for dependent random reads of one padded vector ($150 \text{ GB/s} / 448 \text{ B} = 0.335 \text{ G/s}$). Two independently engineered search builders within 1.3 $\times$ of each

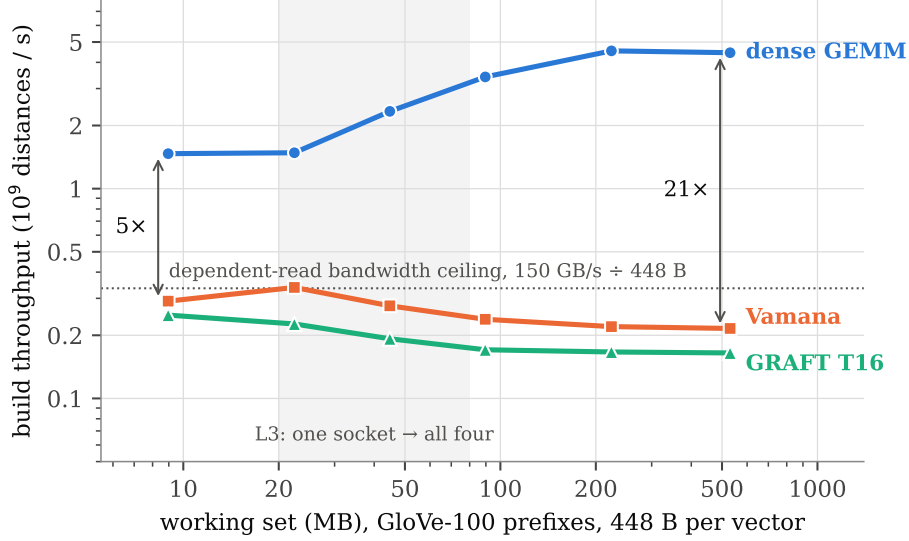


Figure 1: The wall, as a function of the working set (Table 4). The search builders lose a quarter to a third of their throughput between the in-cache and the DRAM-resident regime and never approach the dense kernel; Vamana at full n sits at 65% of the dependent-read bandwidth ceiling. The GEMM curve rises only because its matrix shape improves with n .

other, both 20–27 $\times$ below GEMM at every n , rule out an implementation artifact.

5.3. The model

Write F for the machine’s dense arithmetic rate (flop/s achieved by GEMM), BW for its random-access bandwidth, d for the dimension and $4d$ for the bytes of a vector. A builder whose pair set $\{(p, q)\}$ is fixed before evaluation can tile it, so each loaded vector serves a whole tile: its rate is $F/2d$, and its I/O is $\Theta(\text{pairs}/\sqrt{M})$ for a cache of M words by the Hong–Kung bound [11]. A builder whose next q is a function of the last result cannot tile: each evaluation is one dependent load, its I/O is $\Theta(\text{pairs})$, and its rate is bounded by

$$\min\left(\frac{BW}{4d}, \frac{f_{\text{clk}}}{c_{\text{adapt}}}\right),$$

where c_{adapt} is the per-evaluation bookkeeping in cycles, measured here at 200–250. The ratio of the ceilings is $2F/BW$ for the bandwidth term, independent of d ; with the measured F it is 13 $\times$ on this machine, and the two ceilings

Table 5: Lockstep density of the existing harvest. $|V_i|$ = evaluations per point; union over the block.

harvest	block	B	$ V_i $	$ \bigcup V_i $	density
GloVe T16/ ef_h 400	coherent	178	11,547	384,776	0.028
GloVe T16/ ef_h 400	random	178	12,427	875,260	0.014
GloVe T16/ ef_h 100	coherent	178	3,917	191,922	0.020
SIFT T4/ ef_h 400	coherent	214	2,742	86,375	0.032
SIFT T4/ ef_h 400	random	214	2,823	406,245	0.007

happen to be of similar height (0.34 and 0.29 G/s). This is the roofline [12] with the arithmetic intensity of adaptive evaluation pinned at its minimum, and the old memory wall [13] in its random-access form. The gap widens on newer hardware: wider vector units, quantized GEMM, and GPUs raise F faster than BW and do nothing for c_{adapt} .

Claim 1 (The random-access wall). *A graph construction whose distance evaluations are not coherent, because the distance is a black box, or because the evaluation order depends on previous results, is bounded by random-access throughput, $\min(BW/4d, f_{\text{clk}}/c_{\text{adapt}})$, independently of how few evaluations it performs. Only a decomposable distance evaluated on a pair set fixed in advance is bounded by arithmetic.*

5.4. Can the beam be batched after the fact?

The obvious escape is to run the beams of a block of neighboring points in lockstep and evaluate the block against the union of their candidates as a dense product. Its worth is decided by one number, the *density* $\sum_i |V_i| / (B |\bigcup_i V_i|)$, where V_i is the set of vertices whose distance to point i the beam evaluates: at a $27\times$ kernel advantage, density must exceed 3.7% to break even. We logged V_i for 64 *coherent* blocks (maximal tree-0 SAT subtrees of at most 512 points, median 178) and 64 random blocks of the same size.

Density is 2–3% in every configuration (Table 5), and larger blocks make it worse, since the union saturates at n . Coherence helps only $2\times$ on GloVe and $4.5\times$ on SIFT. The per-phase profile removes the last hope: in expansions 1–10, right after the shared entry set, adjacent points share 1.2% of their evaluations. The roots have degree in the hundreds and the argmin over them differs from point to point, so the beams diverge immediately. What *does* repeat is the target: in coherent blocks, two thirds of all evaluations go

to vertices that at least eight queries of the block also evaluate. A column-batched kernel could load each target once per 8–30 uses and remove the bandwidth term, but not c_{adapt} ; its gain is bounded near $2\times$. Finally, the SAT subtrees are not compact in the scaffold: a coherent block’s one-hop union is $\approx B \times \text{degree}$ (siblings’ neighborhoods barely overlap; a SAT subtree is a cone, not a ball), so the structural candidate sets that would make a dense product exact by construction are not available from the forest either.

The conclusion is not that dense construction is impossible for GRAFT-like designs, but that it cannot be reached by refining the search. It requires the pair set to be fixed before evaluation, which means leaves from a distance-based clustering rather than tree subtrees, plus, for the long-range edges that dense leaves cannot supply and that put PiPNN’s graph 41–45% behind Vamana, a separate and cheap adaptive pass. That is a new construction and a new baseline, outside this paper.

6. Negative result: the scaffold’s degree

Before batching the feedback we tried the direct lever on harvest cost, capping the scaffold’s out-degree at K nearest (GloVe, T32/ ef_h 600, laptop, distance counts). Cap 32 cuts build work by 36% but $d@0.97$ degrades monotonically (+3% at $K=64$, +24% at $K=32$) and completeness falls from 0.590 to 0.543; at matched build cost, lowering ef_h beats capping at every point of the sweep (50.3 G at ef_h 400 gives $d@0.97$ 14,047 where 52.6 G at $K=64$ gives 14,419). Routability needs most of the union’s redundancy, which quantifies the substrate control of [1], and locates the gain of §4 in the substrate’s *staleness*, not its size.

7. Related work

Search-based construction. The proximity-graph lineage runs from the relative neighborhood graph [14] and its sparse navigable relaxations [15] to the navigable small world [16] and HNSW [6], NSG [17] and Vamana/DiskANN [3]; FANNG [9] introduced the occlusion rule that all of them, and GRAFT, use to sparsify a candidate set. These builders are *incremental*: each inserted point searches the graph built so far and mutates it, which is where the feedback loop of §3 comes from and also where their serial dependency chain comes from. ParlayANN [4] batches the *insertions* of Vamana and HCNNG into rounds and obtains determinism and parallel scaling; FGRAFT batches the

feedback into a substrate that begins as a forest union and is frozen within each block, and quantifies what the feedback buys against what the substrate must supply. Surveys of the graph family [18] report neither build-side distance counts nor cost per evaluation, the two quantities this paper measures.

Construction without search. A second lineage fixes its pair sets before evaluating them. NN-descent [19] iterates neighbor-of-neighbor comparisons over a whole graph in rounds; EFANNA [20] and SPTAG [21] seed from randomized k -d or balanced trees; HCNNG [22] and PiPNN [5] cluster hierarchically and brute-force the leaves, PiPNN with dense rank-update GEMMs; CAGRA [23] builds on a GPU from a k -NN graph, and the GPU k -NN graph itself is a blocked product [24]. Every one of these evaluates distances at arithmetic rate because the pairs are known before they are computed, and every one of them needs the distance to decompose into inner products (or, after quantization [25], into table lookups). §5.1 measures the price they pay in graph quality and the price the other lineage pays in throughput.

Metric-space indexing. The rootstock of GRAFT is the spatial approximation tree [2] and its output a half-space proximal graph [26]; both belong to the metric-space setting where the distance is a black box and the number of evaluations is the cost model [27]. The wall of §5 is the statement that this cost model is exact for the black box and off by an order of magnitude for vectors.

Performance models. The bandwidth term is the memory wall [13]; the two-ceiling reading is the roofline model [12]; the reuse argument that separates fixed pair sets from adaptive ones is Hong and Kung’s I/O bound for matrix products [11]. Benchmarks measure indexes by query throughput [8] and explain their difficulty by local intrinsic dimensionality [10]; to our knowledge no prior work applies either performance model to index construction.

8. Conclusion

Incremental construction’s advantage over a frozen substrate is a feedback loop, and the loop is separable from the serial dependency chain that carries it: batching it into eight synchronous blocks recovers the quality of four times as many frozen trees, does less distance work than tuned Vamana, and keeps the graph a pure function of its inputs. The price is a denser substrate and a

higher cost per evaluation, and the result is $1.55\times$ Vamana’s build time on GloVe, $1.73\times$ on SIFT, at the quality of the frozen quality profile.

The larger finding is what bounds that cost. Every search-based builder, including this one and the incremental ones it was measured against, evaluates distances as dependent random accesses and runs an order of magnitude below the machine’s dense arithmetic, most of the gap present with the data in cache. The beam cannot be batched after the fact. For a black-box metric this is simply the cost model of the field and the count of evaluations is the right measure. For vectors it is a wall, and the systems that stand on the other side of it, PiPNN among them, do so by fixing their pair sets before they evaluate them, not by evaluating fewer.

Reproducibility

Code: **graft-ann** (public; the mutation, logging and counter patches are in the paper’s companion repository), the ParlayANN and PiPNN patches (one printed counter; three added counters), the paired driver, the sweep and density scripts, and every manifest and log behind the tables. Seeds are fixed; every GRAFT-family arm produced an identical distance count and completeness in all ten blocks.

References

- [1] E. Chávez, GRAFT: Navigable graphs grown on spatial-approximation rootstock, Information Systems Accepted; INFOSYS-D-26-00905 (2026).
- [2] G. Navarro, Searching in metric spaces by spatial approximation, The VLDB Journal 11 (1) (2002) 28–46.
- [3] S. J. Subramanya, Devvrit, H. V. Simhadri, R. Krishnaswamy, R. Kadekodi, DiskANN: Fast accurate billion-point nearest neighbor search on a single node, in: NeurIPS, 2019, pp. 13748–13758.
- [4] M. D. Manohar, Z. Shen, G. E. Blelloch, L. Dhulipala, Y. Gu, H. V. Simhadri, Y. Sun, ParlayANN: Scalable and deterministic parallel graph-based approximate nearest neighbor search algorithms, in: PPOPP, 2024, pp. 270–285.

- [5] T. Rubel, R. Wen, L. Dhulipala, L. Gottesbüren, R. Jayaram, J. Łacki, PiPNN: Ultra-scalable graph-based nearest neighbor indexing, arXiv:2602.21247 (2026).
- [6] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, *IEEE TPAMI* 42 (4) (2020) 824–836.
- [7] J. Pennington, R. Socher, C. D. Manning, Glove: Global vectors for word representation, in: A. Moschitti, B. Pang, W. Daelemans (Eds.), *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL, ACL, 2014*, pp. 1532–1543.
- [8] M. Aumüller, E. Bernhardsson, A. Faithfull, ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms, *Information Systems* 87 (2020).
- [9] B. Harwood, T. Drummond, FANNG: Fast approximate nearest neighbour graphs, in: *CVPR, 2016*, pp. 5713–5722.
- [10] M. Aumüller, M. Ceccarello, The role of local dimensionality measures in benchmarking nearest neighbor search, *Information Systems* 101 (2021) 101807.
- [11] J.-W. Hong, H. T. Kung, I/O complexity: The red-blue pebble game, in: *STOC, 1981*, pp. 326–333.
- [12] S. Williams, A. Waterman, D. Patterson, Roofline: An insightful visual performance model for multicore architectures, *Communications of the ACM* 52 (4) (2009) 65–76.
- [13] W. A. Wulf, S. A. McKee, Hitting the memory wall: Implications of the obvious, *ACM SIGARCH Computer Architecture News* 23 (1) (1995) 20–24.
- [14] G. T. Toussaint, The relative neighbourhood graph of a finite planar set, *Pattern Recognit.* 12 (4) (1980) 261–268.

- [15] S. Arya, D. M. Mount, Approximate nearest neighbor queries in fixed dimensions, in: SODA, 1993, pp. 271–280.
- [16] Y. Malkov, A. Ponomarenko, A. Logvinov, V. Krylov, Approximate nearest neighbor algorithm based on navigable small world graphs, *Inf. Syst.* 45 (2014) 61–68.
- [17] C. Fu, C. Xiang, C. Wang, D. Cai, Fast approximate nearest neighbor search with the navigating spreading-out graph, *PVLDB* 12 (5) (2019) 461–474.
- [18] M. Wang, X. Xu, Q. Yue, Y. Wang, A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search, *Proc. VLDB Endow.* 14 (11) (2021) 1964–1978.
- [19] W. Dong, M. Charikar, K. Li, Efficient k-nearest neighbor graph construction for generic similarity measures, in: S. Srinivasan, K. Ramamritham, A. Kumar, M. P. Ravindra, E. Bertino, R. Kumar (Eds.), *Proceedings of the 20th International Conference on World Wide Web, WWW 2011, Hyderabad, India, March 28 - April 1, 2011*, ACM, 2011, pp. 577–586.
- [20] C. Fu, D. Cai, EFANNA : An extremely fast approximate nearest neighbor search algorithm based on knn graph, *arXiv:1609.07228* (2016). [arXiv:1609.07228](https://arxiv.org/abs/1609.07228).
- [21] Q. Chen, H. Wang, M. Li, G. Ren, S. Li, J. Zhu, J. Li, C. Liu, L. Zhang, J. Wang, [SPTAG: A library for fast approximate nearest neighbor search](https://github.com/Microsoft/SPTAG), GitHub (2018).
URL <https://github.com/Microsoft/SPTAG>
- [22] J. A. Vargas Muñoz, M. A. Gonçalves, Z. Dias, R. da Silva Torres, Hierarchical clustering-based graphs for large scale approximate nearest neighbor search, *Pattern Recognition* 96 (2019).
- [23] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, Y. Wang, CAGRA: highly parallel graph construction and approximate nearest neighbor search for gpus, in: *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*, IEEE, 2024, pp. 4236–4247.

- [24] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, *IEEE Transactions on Big Data* 7 (3) (2021) 535–547.
- [25] H. Jégou, M. Douze, C. Schmid, Product quantization for nearest neighbor search, *IEEE Trans. Pattern Anal. Mach. Intell.* 33 (1) (2011) 117–128.
- [26] E. Chávez, S. Dobrev, E. Kranakis, J. Opatrny, L. Stacho, H. Tejada, J. Urrutia, Half-space proximal: A new local test for extracting a bounded dilation spanner of a unit disk graph, in: J. H. Anderson, G. Prencipe, R. Wattenhofer (Eds.), *Principles of Distributed Systems*, 9th International Conference, OPODIS 2005, Pisa, Italy, December 12-14, 2005, Revised Selected Papers, Vol. 3974 of *Lecture Notes in Computer Science*, Springer, 2005, pp. 235–245.
- [27] E. Chávez, G. Navarro, R. A. Baeza-Yates, J. L. Marroquín, Searching in metric spaces, *ACM Comput. Surv.* 33 (3) (2001) 273–321.